

ab_slot_switch

ab_slot_switch.sh — user guide

Revision: 1 **Last modified:** 2026-06-11T00:00:00Z

Companion documentation (§11.4.18) for tests/emulator/ab_virt/
ab_slot_switch.sh.

Overview

ab_slot_switch.sh proves the **real, bootloader-driven A/B slot switch** of the RK3588 A/B-virt emulator on an Apple-Silicon host: it boots the SAME 2-slot GPT disk (ab_disk.img) twice with the SAME real u-boot.bin, differing ONLY in the U-Boot BOOT_ORDER environment variable, and asserts that each boot routes the kernel root= to a DIFFERENT physical rootfs partition AND that the guest userspace observably reports the slot it was actually booted into.

- **Run A** — BOOT_ORDER="A B" → head slot A → root=/dev/vda2 → guest / etc/slot_id=A + kernel cmdline helix_slot=A.
- **Run B** — BOOT_ORDER="B A" → head slot B → root=/dev/vda3 → guest / etc/slot_id=B + kernel cmdline helix_slot=B.

The proof is the **contrast** (§11.4.115 RED→GREEN): a fake / no-op switch would report the same slot in both runs (RED); a real switch reports A then B (GREEN). Both consoles are captured in full — real getty login + post-login command output, a live shell, not a frozen frame (§11.4.107).

This is the OTA A/B primitive (the RAUC / Mender BOOT_ORDER convention) running under a real **U-Boot 2024.01** + **QEMU -machine virt + HVF** — not a mock. It is the slot-switch phase that assemble_ab_disk.sh prepared the disk for.

Proven status (FACT, §11.4.6): this slot switch has run GREEN. Evidence: docs/qa/20260611T094958Z-ab-slot-switch/ — verdict.txt records Run A: HELIX_SLOTID=A HELIX_ROOTDEV=/dev/vda2, Run B: HELIX_SLOTID=B HELIX_ROOTDEV=/dev/vda3, both rc=0, u-boot.bin: U-Boot 2024.01, Verdict: PASS; full transcripts in consoleA.log + consoleB.log.

What is NOT yet proven (§11.4.6 — honest boundary): this test proves bootloader-driven slot routing + in-guest observability ONLY. It does NOT prove persistent saveenv BOOT_ORDER across a power-cycle (the BOOT_ORDER is set

interactively per run, not persisted), RAUC dm-verity / AVB-style slot integrity, or automatic bootcount-driven auto-rollback. Those are later PWUs; no claim is made here that they work.

Prerequisites

- A completed `build_image.sh` run: `out/.ok` present and `out/images/u-boot.bin` non-empty (the script aborts with exit 3 otherwise).
- A completed `assemble_ab_disk.sh` run: `out/.disk_ok` present and `out/images/ab_disk.img` non-empty (exit 3 otherwise).
- `qemu-system-aarch64` on `PATH` with HVF support.
- `expect` on `PATH`.
- `HELIX_AB_ROOT_PW` must match the password baked into the image by `build_image.sh` (default `helixota`).

Usage examples

```
# Run the full A/B slot-switch proof (after build_image.sh +
    assemble_ab_disk.sh
# have produced GREEN out/.ok + out/.disk_ok).
tests/emulator/ab_virt/ab_slot_switch.sh

# With a non-default root password (must match the built image).
HELIX_AB_ROOT_PW=helixota tests/emulator/ab_virt/ab_slot_switch.sh
```

Inputs

Input	Source	Default / requirement
<code>HELIX_AB_ROOT_PW</code>	<code>env</code> → <code>ROOT_PW</code>	<code>helixota</code>
<code>out/.ok</code> , <code>out/images/u-boot.bin</code>	filesystem (from <code>build_image.sh</code>)	required, non-empty
<code>out/.disk_ok</code> , <code>out/images/ab_disk.img</code>	filesystem (from <code>assemble_ab_disk.sh</code>)	required, non-empty

The run id is derived from a UTC timestamp: `<YYYYmmddTHMMSSZ>-ab-slot-switch`.

Outputs

All under `docs/qa/<run-id>/` (the captured evidence, §11.4.83):

Output	Meaning
<code>docs/qa/<run-id>/consoleA.log</code>	full QEMU transcript of Run A (<code>BOOT_ORDER="A B"</code>)
<code>docs/qa/<run-id>/consoleB.log</code>	full QEMU transcript of Run B (<code>BOOT_ORDER="B A"</code>)

Output	Meaning
docs/qa/<run-id>/ consoleA.log.driver, consoleB.log.driver	the expect driver's own stdout/stderr per run
docs/qa/<run-id>/drive.exp	the emitted expect driver source (one Tcl quoting level)
docs/qa/<run-id>/verdict.txt	distilled verdict: u-boot version, both rc's, the per-run slot_id + rootdev, Verdict: PASS/FAIL
stdout PASS/FAIL lines	one [PASS]/[FAIL] per anti-bluff assertion + a final RESULT line
exit code	0 = PASS, 1 = FAIL, 3 = precondition abort

Side-effects

- Spawns two transient qemu-system-aarch64 guests (one per run); each exits on guest poweroff or the expect timeout (self-cleaning, §11.4.14).
- Creates the docs/qa/<run-id>/ evidence directory.
- Does **not** modify the disk artifact (ab_disk.img is attached read-write but both guests are powered off cleanly; BOOT_ORDER is set interactively and never saveenv'd, so the on-disk U-Boot env is not persisted).

Internal behaviour

1. set -u + set -o pipefail; resolve SCRIPT_DIR, repo root, root password, u-boot.bin/ab_disk.img paths, run id, and the evidence dir; mkdir -p the evidence dir.
2. **Preconditions** (each → exit 3 on failure): out/.ok present; out/.disk_ok present; u-boot.bin non-empty; ab_disk.img non-empty; qemu-system-aarch64 found; expect found.
3. Emit a deterministic expect driver to a temp .exp file via a single-quoted heredoc — so there is exactly ONE quoting level (Tcl) and guest-shell command substitution \\$(...) reaches the file verbatim, ensuring the GUEST shell (not Tcl) runs it (§11.4.1 — the FAIL must be a product defect, never a script bug).
4. For each run (A then B) the driver (timeout 150s):
 - spawns QEMU -M virt -accel hvf -cpu host -smp 2 -m 512 -nographic -bios <u-boot.bin> -drive file=<ab_disk.img>, if=virtio, format=raw;
 - interrupts U-Boot autoboot, reaches the => U-Boot prompt;
 - setenv BOOT_ORDER <order> (the OTA agent's head-slot convention)
 - printenv BOOT_ORDER;
 - load virtio 0:1 0x40400000 boot.scr then source 0x40400000 (runs uboot_ab/boot.cmd, which selects the head slot and bootis the kernel with root=/dev/vda<part> + helix_slot=<slot>);
 - waits for buildroot login: (tolerating interleaved kernel-console noise), login as root with one retry if the prompt re-displays;

- from inside the guest shell prints `HELIX_SLOTID=$(cat /etc/slot_id)`, `HELIX_ROOTDEV=$(findmnt -no SOURCE /)`, `HELIX_CMDLINE=$(cat /proc/cmdline)`, and a `HELIX_DONE_<A|B>_MARK` post-login sentinel; then `poweroff`.
 - on any missing prompt the driver prints `HELIX_DRIVER_FAIL: ...` and exits 2; a `poweroff` timeout is a non-fatal note (`HELIX_DRIVER_NOTE`, exit 0). The transcript is captured via `log_file` into `consoleA.log` / `consoleB.log`.
5. **Anti-bluff assertions** — `grep` the captured consoles; the proof is the contrast:
- Run A must show `HELIX_SLOTID=A`, `HELIX_ROOTDEV=/dev/vda2`, `helix_slot=A`, `HELIX_DONE_A_MARK` (live shell), AND must NOT show `HELIX_SLOTID=B`.
 - Run B must show `HELIX_SLOTID=B`, `HELIX_ROOTDEV=/dev/vda3`, `helix_slot=B`, `HELIX_DONE_B_MARK`, AND must NOT show `HELIX_SLOTID=A` (the switch is real, not a no-op).
6. Write `verdict.txt` (u-boot version, both rc's, per-run `slot_id` + `rootdev`, Verdict: PASS/FAIL). If every assertion holds, print `RESULT: PASS` + the evidence path and exit 0; otherwise print `RESULT: FAIL` (with the `qemu/expect` rc's) and exit 1.

Edge cases

- **out/.ok / out/.disk_ok absent, or u-boot.bin / ab_disk.img missing** → exit 3 (run `build_image.sh` then `assemble_ab_disk.sh` first).
- **QEMU or expect not installed** → exit 3.
- **No autoboot / U-Boot prompt / boot.scr load / login / password / shell prompt within timeout** → the driver emits a `HELIX_DRIVER_FAIL` line and exits 2; the corresponding contrast assertions then FAIL and the script exits 1.
- **Login re-prompts** → the driver retries the login cycle once (a third re-prompt is a `HELIX_DRIVER_FAIL`).
- **Poweroff times out** → treated as a non-fatal `HELIX_DRIVER_NOTE` (driver exits 0); the verdict still depends solely on the captured-evidence contrast assertions.
- **A no-op / fake switch (same slot both runs)** → the contrast + negative (nchk) assertions FAIL → Verdict: FAIL, exit 1.

Related scripts

- `tests/emulator/ab_virt/assemble_ab_disk.sh` — produces the 2-slot GPT `ab_disk.img` + compiled `boot.scr` this script boots. See `docs/scripts/assemble_ab_disk.md`.
- `tests/emulator/ab_virt/build_image.sh` — produces the kernel Image
 - `rootfs.ext2` + `u-boot.bin`. See `docs/scripts/build_image.md`.
- `tests/emulator/ab_virt/boot_smoke.sh` — base-image boot smoke (the proven foundation, single-slot). See `docs/scripts/boot_smoke.md`.
- `uboot_ab/boot.cmd` + `uboot_ab/README.md` — the U-Boot A/B boot script source (compiled to `boot.scr`) this test sources and the slot-selection contract it must match.
- `docs/research/rk3588_emulator/REPORT.md` — the emulator design report (PWU-AB-1).

Last verified date

2026-06-11 — documented against the script as committed (read, not inferred). The bootloader-driven A/B slot switch is PROVEN GREEN; evidence at docs/qa/20260611T094958Z-ab-slot-switch/verdict.txt (Run A slot A / vda2, Run B slot B / vda3, both rc=0, U-Boot 2024.01). Persistent saveenv power-cycle, RAUC dm-verity, and auto-rollback are later PWUs and remain UNPROVEN.